Octopus

Lend your laptop to your team's CI, over Logos Delivery

Hackathon - Logos Messaging Offsite 2026

Logos Delivery

Fabiana Cecin · Tanya Stubbs · Ivan Folgueira Bande

THE PROBLEM

CI minutes cost money, and the laptops sit idle

- Hosted runners bill by the minute; macOS minutes cost the most
- Every developer has a fast machine that is idle most of the day
- Self-hosted runners exist, but:
  - registering one needs repo admin rights
  - an always-on coordinator needs a GitHub admin token on a server
- We wanted: team machines run the CI, with no server and no stored admin token

GitHub-hosted runner
paid per minute

Team laptop
idle, already trusted

Coordinator server
holds an admin token

THE IDEA

Logos Delivery is the control plane

- No coordinator: the team machines and the CI job meet on a Logos Delivery topic
- Discovery, job assignment, results and live logs all travel over the network
- GitHub only proves who is asking (OIDC token) — it never gets a runner registration
- Anything no machine takes, and Windows, falls back to a GitHub-hosted runner

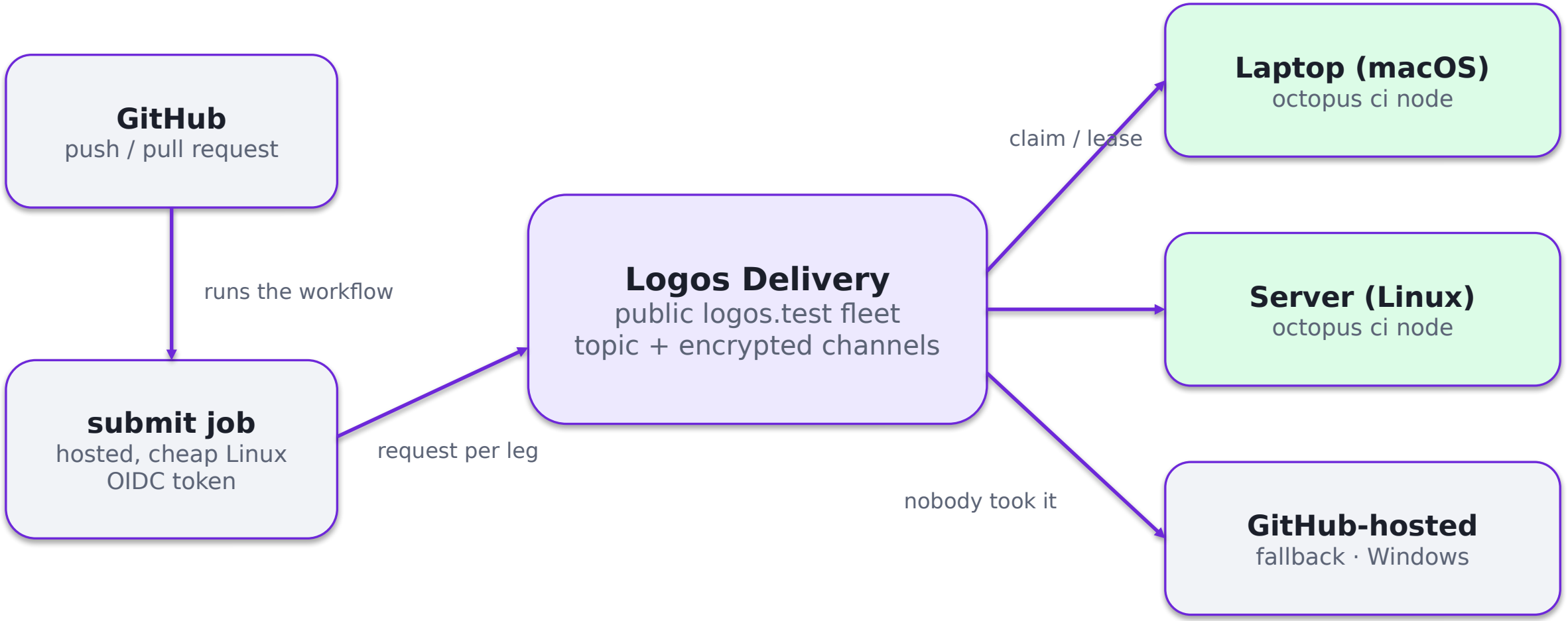
Discovery
who is online, which OS

Assignment
claim · lease · takeover

Results & logs
encrypted reply channel

HOW IT WORKS

How a CI run reaches a laptop



Nodes verify the token against GitHub's public keys, rank themselves (rendezvous hash), claim, run .octopus/ci.sh, and stream the log back. The submit job prints it and gates the PR.

Who may run what

- Every request carries a GitHub OIDC token
  - checked against GitHub's public keys; one-time use
  - pinned workflow, allowed repo, no fork PRs
- Team membership is a signed chain
  - team key → member list (GitHub logins, per repo/org)
  - → SSH signature of each node, checked on github.com/<login>.keys
- No admin token on any server
- Jobs can run in a throwaway container
  - sees only the checkout; no gh login, SSH keys or home
  - read-only root, no capabilities, resource limits
- Honest limits
  - a member's machine is trusted to report the truth
  - container on Linux only; macOS needs a VM

DEMO

Live demo

1	`octopus ci status`	this laptop's proof, team key and allowlist — its only setup
2	`git push`	a one-line change starts the workflow on GitHub
3	`submit job joins the fleet`	publishes one request per leg on Logos Delivery
4	`CI leg started …`	the laptop claims the macOS leg and runs .octopus/ci.sh
5	`[macos] …`	its output streams live into the GitHub Actions log
6	`job summary`	macOS: this laptop · Linux: server node or hosted · Windows: hosted

WHY IT WORKS

What Logos Delivery gave us

Reach

Nodes behind home NATs talk through the public fleet: no ports to open, no server

Discovery

One pub/sub topic: machines announce, CI jobs ask, everyone hears

Private channels

Reliable, end-to-end encrypted channels for results and live logs

Fast join

A fresh node joins the fleet and exchanges messages in about 4 seconds

We used it on itself

- Octopus runs its own CI through Octopus
- We installed it as new users and filed what hurt
- Fixed during the hackathon:
  - install ~9 min → under a minute (no submodule clones)
  - library 279 MB → 68 MB (stripped)
  - TRACE flood ~360 lines/s → INFO by default
  - stop took 24 s → immediate
  - clear errors for missing gh login and tokens
- Still open:
  - a node keeps ≤ 5 peers per IP (library limit)
  - nightly end-to-end test needs its repo set up
- Three ways to use it, one CLI:
  - octopus ci init — join the team CI
  - octopus init — a plain runner for a repo
  - coordinator mode — one-use JIT runners

Status and next

- v0.4.0 released: pip wheel, prebuilt libraries for Linux x86_64 and macOS arm64
- octopus update upgrades a machine and restarts its service
- Next:
  - “drivers”: GitHub today; GitLab or a plain “run this” later
  - a macOS VM sandbox, like the Linux container
  - more team machines, and measuring cost and speed against hosted runners

Thank you

github.com/logos-messaging/octopus

Fabiana Cecin · Tanya Stubbs · Ivan Folgueira Bande